The default value is `20s` (20 seconds), which is probably too much for a standard cloud native application. In the context of highly distributed systems in the cloud, we probably don't want to wait more than a couple of seconds and risk a cascading failure due to a Tomcat instance hanging for too long. Something like `2s` would be better. You can also use the `server.tomcat.keep-alive-timeout` property to configure how long to keep a connection open while waiting for new HTTP requests.

Listing 3.2 Configuring timeouts for Tomcat

```
server:
  port: 9001
  tomcat:
    connection-timeout: 2s
    keep-alive-timeout: 15s
```

THREAD POOL

Tomcat has a pool of threads that process requests, following the thread-per-request model. The number of available threads will determine how many requests can be handled simultaneously. You can configure the maximum number of request processing threads through the `server.tomcat.threads.max` property. You can also define the minimum number of threads that should always be kept running (`server.tomcat .threads.min-spare`), which is also how many threads are created at startup.

Determining the best configuration for a thread pool is complicated, and there's no magic formula for computing it. Resource analysis, monitoring, and many trials are usually necessary to find a suitable configuration. The default thread pool can grow up to `200` threads and has `10` worker threads always running, which are good starting values in production. In your local